

CS605 Short Notes final term

Lec 19 to 45

BY vuonlinehelp.blogspot.com

YOUTUBE LINK:

WEBSITE LINK:

[HTTPS://VUONLINEHELP.BLOGSPOT.COM/](https://vuonlinehelp.blogspot.com/)

IMPORTANT NOTE

You should never copy paste the same answer which we are providing. Please do make some changes in the solution file. If you appreciate our work, please share our website vuonlinehelp.blogspot.com with your friends.

آپ کو کبھی بھی وہی جواب کاپی پیسٹ نہیں کرنا چاہیے جو ہم فراہم کر رہے ہیں۔ براہ کرم حل فائل میں کچھ تبدیلیاں کریں۔ اگر آپ ہمارے کام کی تعریف کرتے ہیں تو براہ کرم ہماری ویب سائٹ vuonlinehelp.blogspot.com کو اپنے دوستوں کے ساتھ شیئر کریں

Lecture No. 18 Risk analysis and management

Risk analysis and management

Analysis and management of project risks is also a very important activity that a project manager must perform in order to **improve the chances for the project**. Robert Charette defines risk as follows:

“Risk concerns future happenings. Today and yesterday are beyond active concern. The question is, can we, therefore, by changing our action today create an opportunity for a different and hopefully better situation for ourselves tomorrow. This means, second, that risk involves change, such as changes in mind, opinion, action, or places. ... [Third,] risks involve choice, and the uncertainty that choice itself entails.”

Risk analysis and management involves addressing the following concerns:

1. **Future** – what risks might cause the project to go awry
2. **Change** – what change might cause the risk to strike, How changes in requirements, technology, personnel and other entities connected to the project affect the project
3. **Choice** – what options do we have for each risk

In Peter Drucker words:

While it is futile to try to eliminate risk, and questionable to try to minimize it, it is essential that the risk taken be the right risk there are **two basic risk management philosophies, reactive and proactive**.

- **Reactive**
 - Indiana Jones school of risk management
 - Never worrying about problems until they happened, and then reacting in some heroic way – Indiana Jones style.
- **Proactive**
 - Begins long before technical work starts
 - Risks are identified, their probability and impact are analyzed, and they are ranked by importance.
 - Risk management plan is prepared
 - Primary objective is to avoid risk
 - Since all risks cannot be avoided, a contingency plan is prepared that will enable it to respond in a controlled and effective manner

Risk characteristics:

A risk has two characteristics:

- **Uncertainty** – the risk may or may not happen
- **Loss** – if the risk becomes a reality, unwanted consequences or losses will occur.

Lecture No. 19 Types of Risks

Project risks

- Impact schedule and cost
- Includes budgetary, schedule, personnel, resource, customer, requirement problems

Technical risks

- Impact the quality, timelines, and cost
- Implementation may become difficult or impossible
- Includes design, implementation, interface, verification and maintenance problems
- Arise due to leading edge technology

Business risks

- Marketability
- Alignment with the overall business strategy
- How to sell
- Budget or personnel commitments

Risk Identification

- Identify known and predictable risks
- Generic risks and product specific risks
- Generic risks – threats to every project
- Product specific risks – what special characteristics of this project may threaten your project plan?
- Risk item checklist

Risk Item Checklist

- Product size
- Business impact
- Customer characteristics
- Process definition
- Development environment
- Technology to be built
- Staff size and experience
- Questions relevant to each of the topics can be answered for each software project

Assessing Overall Project Risks

- Have top software and customer managers formally committed to support the project?
- Are end-users committed to the project and the system/product to be built?

- Are requirements fully understood?
- Have customers been involved fully in requirement definition?
- Do end-users have realistic expectations?
- Does the software team have right mix of skills?
- Are project requirements stable?
- Does the project team have experience with the technology to be implemented?
- Is the number of people on the project team adequate to do the job?

Risk components and drivers

Performance risks

Degree of uncertainty that the product will meet its requirements and be fit for its intended use

Cost risks

The degree of uncertainty that the project budget will be maintained

Support risks

Resultant software will be easy to correct, enhance, and adapt.

Schedule risks

- Product schedule will be maintained

Four impact categories

- Negligible
- Marginal
- Critical
- Catastrophic

Row 1—Consequences of errors		Row 2—Failure to achieve desired outcome		
	Performance	Support	Cost	Schedule
Catastrophic	Failure to meet the requirements will result in mission failure		Results in increased cost and schedule delays. Expected value in excess of \$500K	
	Significant degradation	Non-responsive or unsupportable	Budget overrun likely	Unachievable
Critical	Would degrade performance to a point where mission success is questionable		Results in operational delays and or increased cost with expected value of \$100K-\$500K	
	Some reduction in technical performance	Minor delays	Possible overrun	Possible slippage
Marginal	Result in degradation of secondary mission		Expected value <\$100K	
	Small reduction	Responsive	Sufficient financial resources	Realistic
Negligible	Inconvenience		Minor	
	No reduction	Supportable	Budget under run possible	Achievable

Risk Projection

Risk projection is concerned with risk estimation. It attempts to rate risks in two ways: likelihood and consequences. There are four risk project activities. These are:

- Establish a scale that reflects the perceived likelihood of risk
- Delineate the consequences
- Estimate impact
- Note the overall accuracy of risk projection

1: Catastrophic

2: Critical

3: Marginal

4: Negligible

Risk mitigation, monitoring, and management plan

Risk	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End-users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools				
Staff inexperienced	DE	80%	3	
Staff turnover will be high	ST	30%	2	
	ST	60%	2	

Lecture No. 20 Assessing Risk Impact

Assessing Risk Impact

Assessment of risk impact is a non-trivial process. Factors affecting the consequences are the nature, scope, and timing.

For each risk an exposure is calculated as follows:

$$RE = \text{Probability of the risk} \times \text{Cost}$$

Example

Risk Identification

Only 70% of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Risk Probability

80% likely

Risk impact

60 reusable software components were planned. If only 70% can be used, 18 components would have to be developed from scratch. Since the average component will cost 100,000, the total cost will be 1,800,000.

$$RE = 0.8 * 1,800,000 = 1,440,000$$

Risk Refinement

Analyze and refine the risk into more detailed risks.

Mitigation/Monitoring

Contact 3rd party to determine conformance with design standards. Press for interface standard completion; consider component structure when deciding on interface protocol. Check to determine if language support can be acquired.

Management/Contingency Plan

RE computed to be 1,440,000. Allocate this amount within project contingency cost. Develop revised schedule assuming 18 additional components will have to be custom-built. Allocate staff accordingly

Risk Mitigation, Monitoring, and Management (RMMM)

Assists the project team in developing strategy for dealing with risk an effective strategy must consider:

- Risk avoidance
- Risk monitoring
- Risk management and contingency plan

Remember 'Avoidance is always the best strategy'

Risk management and contingency plan

Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality.

- Risk has become a reality – some people announce that they will be leaving
- If mitigation strategy has been followed, backup is available, information has been documented, and knowledge has been dispersed
- Temporarily refocus and readjust resources
- People who are leaving are asked to stop all work and ensure knowledge transfer

Risk mitigation and contingency is a costly business. It is therefore important to understand that for RMMM plan, a cost/benefit analysis of each risk must be carried out. The Pareto principle (80-20 rule) is applicable in this case as well 20% of the identified risk account for 80% of the potential for project failure.

Mitigation Strategy for Reducing Turnover

- Meet with current staff to determine causes for turnover (e.g. poor working conditions, low pay, and competitive job market)
- Mitigate those causes that are under our control before the project starts
- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave
- Organize project teams so that information about each development activity is widely dispersed
- Define documentation standards and establish mechanisms to be sure that documents are developed in a timely manner (to ensure continuity)
- Conduct peer reviews of all work (so that more than one person is up to speed)
- Assign a backup staff member for every critical technology

Monitoring Risk Factors

- General attitude of team members based on project pressures
- The degree to which the team is jelled
- Interpersonal relationships among team members
- Potential problems with compensation and benefits
- The availability of jobs within the company and outside it.

Lecture No. 21 Software Projects Scheduling and Monitoring

Software Project Scheduling and Monitoring

Software project scheduling is the next task to be performed by the PM. It is important to note once again that in the reasons for project failure, unrealistic deadline and underestimate of effort involved in the project are two of the most important reasons for project failure. Therefore, a good schedule estimate would increase the chances of the success of the project.

Monitoring effectiveness of risk mitigation steps

The PM should monitor documents carefully to ensure that each can stand on its own and that each imparts information that would be necessary if a newcomer were forced to join the software team somewhere in the middle of the project.

Why Projects Fail?

- An unrealistic deadline is established
- Changing customer requirements
- Ambiguous/incomplete requirements
- An honest underestimate of effort
- Predictable and/or unpredictable risks
- Technical difficulties
- Miscommunication among project staff
- Failure in project management

Basic Concepts

Projects fall behind schedule one day at a time (Fred Brooks).

Project manager's objective

- Identify and define all project tasks
- Build a network that depicts their interdependencies
- Identify the tasks that are critical within the network
- Track their progress to ensure delay is recognized one day at a time

The schedule must be fine grained.

Software Project Scheduling

- Software project scheduling is an activity that distributes estimated effort across the planned project duration by allocating the effort to specific software engineering tasks
- Macroscopic and detailed schedule
- Delivery date driven schedule vs. schedule for resource optimization

Basic Principles

- Compartmentalization
- Interdependency
- Time allocation
- Effort validation
- Defined responsibilities
- Defined outcomes

- Defined milestones

Compartmentalization

The project must be compartmentalized into a number of manageable activities and tasks. To accomplish compartmentalization, both the product and process are decomposed.

Interdependency

The interdependency of each compartmentalized activity or task must be determined. Some tasks must occur in sequence while others can occur in parallel. Some activities cannot commence until the work product produced by another is available.

Time allocation

Each task to be scheduled must be allocated some number of work units (e.g. person-days of effort). In addition, each task must be assigned a start date and an end date which is a function of the interdependencies and number of resources.

Effort validation

Every project has a defined number of staff members. As time allocation occurs, the project manager must ensure that no more than the allocated number of people has been scheduled at any given time.

Defined responsibilities

Every task should be assigned to a specific team member.

Defined outcomes

Every task should have a defined outcome, normally a work product.

Defined milestones

Every task or group of tasks should be associated with a project milestone.

Lecture No. 22 Relationship between people and effort

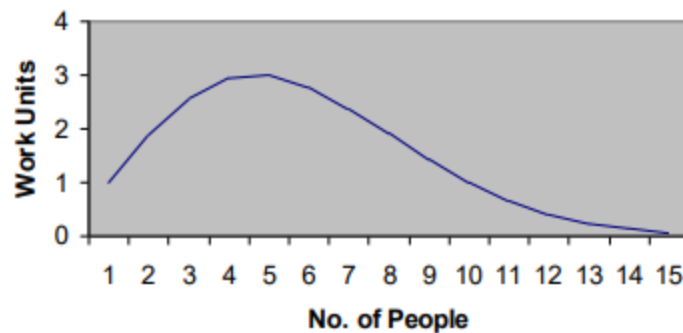
Relationship between people and effort

The relationship between the number of people and time to develop an application is not linear. It is not as simple as a 120 man-day project can be developed by 1 person working for 120 days or 120 people working for 1 day. The communication and coordination overhead plays a very significant role.

Communication and coordination overhead

- Number of Channels $C = N(N-1)/2$
- Communication overhead per channel k
- Work accomplished $W = (1-k)^C \times N$

Work Accomplished with only 5% Communication Overhead per Channel



Task Set Definition

Process model → task sets (SE work tasks, milestones, and deliverables). Enable a software team to define, develop, and support. Each software process should define a collection of task sets, each designed to meet the needs of different types of projects.

Project Types and Rigor

Different types of projects and different degree of rigor.

Types of projects

- Concept development projects
- New application development
- Application enhancement
- Application maintenance
- Reengineering projects

Degree of Rigor

Casual

- All process framework activities are applied, but only a minimum task set is required
- Reduced umbrella tasks, reduced documentation
- Basic principles of SE are followed

Structured

- Process framework is applied
- Appropriate framework activities, related tasks, and umbrella activities (to ensure high quality) applied
- SQA, SCM, documentation, and measurement conducted in streamlined manner.

Strict

- Full process
- All umbrella activities are applied
- Robust work products

Quick Reaction

- Because of emergency, only task essential for maintaining good quality are applied.
- Back-filling

Defining Adaptation Criteria

- Size of the project
- Number of potential users
- Mission criticality
- Application longevity
- Stability of requirements
- Ease of customer/developer communication
- Maturity of application technology
- Performance constraints
- Embedded or non-embedded characteristics
- Project staff
- Reengineering factors

Computing a Task Set Selector (TSS) value

- Adaptation criteria, grade, weight, entry point multiplier.
- Assign appropriate grades (1-5) to each adaptation criteria.
- Review weighting factor and modify if needed. The ranges should be between 0.8 to 1.2.

Multiply the grade entered by the weight and by entry point multiplier for the type of project undertaken. The entry point multiplier takes a value of 0 or 1 and indicates the relevance of the adaptation criterion to the project type.

Product = grade x weight x entry point multiplier

TSS = average of all the entries in the product column

TSS	Degree of Rigor
< 1.2	Casual
1.0 – 3.0	Structured
> 2.4	Strict

Lecture No. 23 Task Network

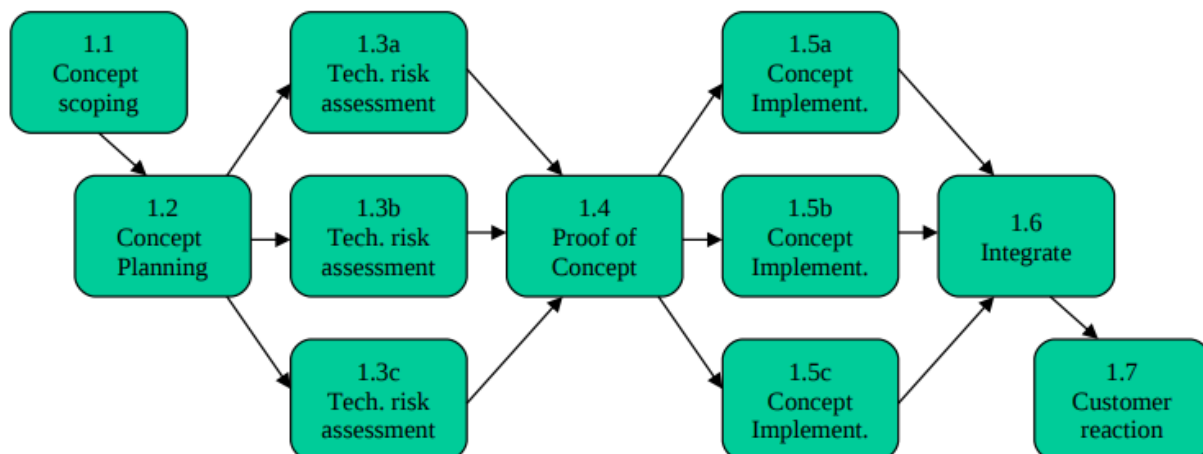
The principles studied above to develop a plan for a concept development project. In this case the potential for new technology or new business idea is explored.

Major tasks include:

- Concept scoping
- Preliminary concept planning
- Technology risk assessment
- Proof of concept
- Concept implementation
- Customer reaction to concept

Defining a Task Network

Develop a task network to determine the sequence in which these activities need to be performed. This will ultimately lead to the time required to complete the project (to be discussed later). The following diagram shows the task network for the above project.



Lecture No. 24 Scheduling

Scheduling

Prepare a schedule for the project. For this we use two techniques known as:

- Program evaluation and review techniques (PERT)
- Critical Path Method (CPM)

These are quantitative tools that allow the software planner to determine the critical path the chain of tasks that determines the duration of the project and establish most likely time estimates for individual tasks by applying statistical models. They also help the planner to calculate boundary times that define a time window for a particular task.

Boundary time

The boundary time defines the following parameters for a project:

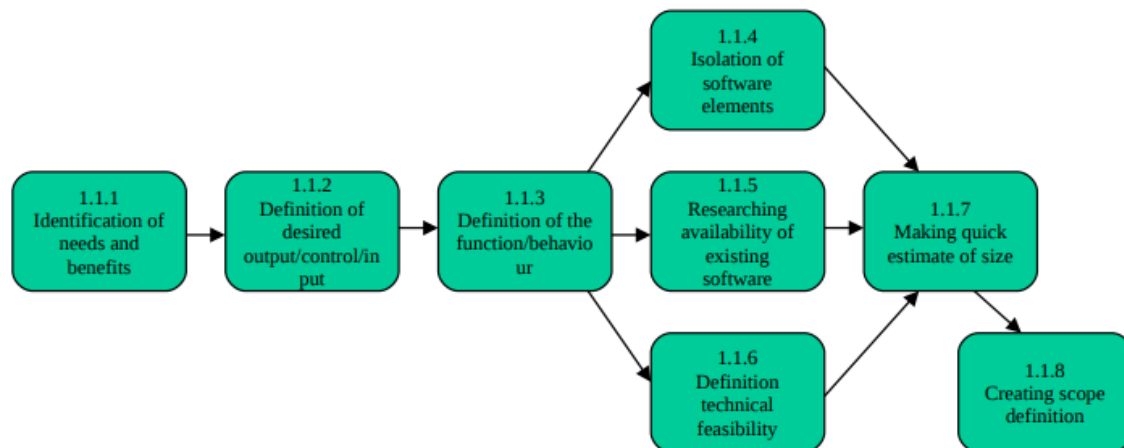
- The earliest time that a task can begin when all preceding tasks are completed in the shortest possible time
- The latest time for task initiation before the minimum project completion time is delayed
- The earliest finish
- The latest finish
- The total float the amount of surplus time or leeway allowed in scheduling tasks

So that the network critical path is maintained on schedule

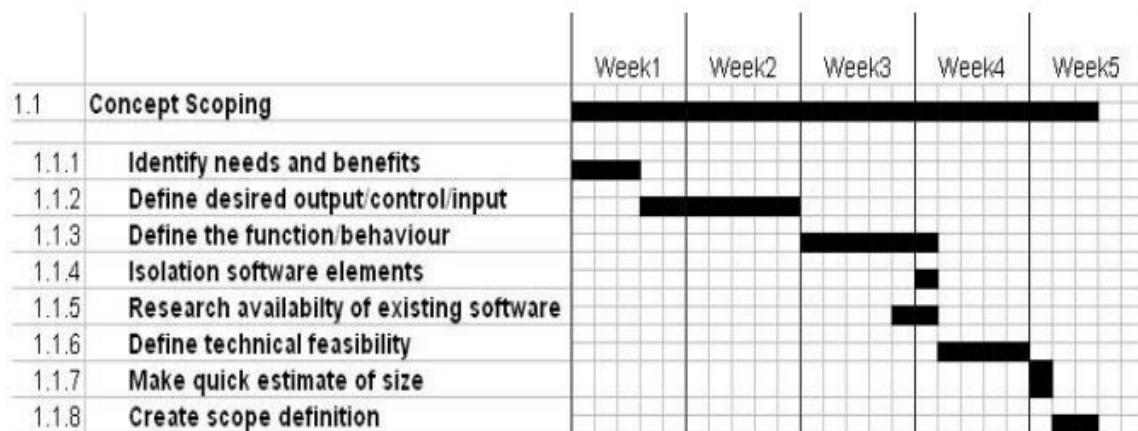
Timeline Chart

To develop the schedule for a project, time required for each activity in the Task Network is estimated. This analysis and decomposition leads to the development of a Timeline or Gantt Chart for the project which portrays the schedule for the project. As an example, let us assume that Concept Scoping (the first task in the above list) is further subdivided into the following sub-tasks with the associated estimated time requirements:

1. Identification of needs and benefits (3 days)
2. Definition of desired output/control/input (7 days)
3. Definition of the function/behavior (6 days)
4. Isolation of software elements (1 day)
5. Researching availability of existing software (2 days)
6. Definition technical feasibility (4 days)
7. Making quick estimate of size (1 day)
8. Creating scope definition (2 days)



Now converted in the following schedule in the form of a Gantt Chart. Note that, the concept of boundary time allows us to schedule Task Numbers 1.1.4 and 1.1.5 anywhere along Task Number 1.1.3. The actual time is determined by the project manager is based upon the availability of resources and other constraints. Each task is further subdivided in sub-tasks in the same manner until the schedule for the complete project is determined.



Lecture No. 25 Project Tracking

Project Tracking

A schedule is meaningless if it is not followed and tracked. Tasks and milestones defined in a project schedule must be tracked and controlled as project proceeds. Tracking methods include:

- Periodic project status meetings
- Evaluating the results of all reviews
- Determine whether project milestones have been accomplished by the scheduled date
- Comparing actual start date to planned start date
- Informal meetings with the practitioners
- Using earned value analysis
- Error tracking

The last two techniques are discussed in further detail in the following paragraphs:

Earned Value Analysis

Earned Value Analysis or EVA is a quantitative technique for assessing the progress of a project. The earned value system provides a common value scale for every software task, regardless of the type of work being performed. The total hours to do the whole project are estimated, and every task is given an earned value based on the estimated percentage of the total. In order to do the EVA, the budgeted cost of work schedule (BCWS) is

Let

$BCWS_i$ = effort (person-days etc) for task_i

BCWS is then the Progress so far – add all $BCWS_i$ so far.

Now

BAC = budget at completion = $\sum BCWS_i$

Now if BCWP is the Budgeted Cost of Work Perform, then

Schedule performance index $SPI = BCWP/BCWS$

Schedule variance $SV = BCWP - BCWS$

SPI close to 1 indicates efficient execution.

Similarly

Percent scheduled for completion = $BCWS/BAC$

Percent complete = $BCWP/BAC$

Actual cost work performed ACWP

Cost performance index $CPI = BCWP/ACWP$

Cost variance $CV = BCWP - ACWP$

Now, value of CPI close to 1 means project is within its defined budget. Therefore, by using SPI and CPI we estimate how the project is progressing. If we have these values close to 1, it means that we have had good estimates and the project is under control.

Time Boxing

Used in severe deadline pressure. Use incremental strategy. Tasks associated with each increment are time-boxed. Schedule for each task is adjusted by working backward from the delivery date. A box is put around each task when a task hits the boundary of the box, work stops and next task begins 90-10 rule rather than becoming stuck on the 10% of a task, the product proceeds towards the delivery date.

Lecture No. 26 Error Tracking

Error Tracking

Error tracking can also be used to estimate the progress of the project. In this case we track errors in work products (requirement specifications, design documents, source code etc) to assess the status of a project. The process works as follows:

We collect error related metrics over many projects and determine our defect removal efficiency in the following manner:

- Defect removal efficiency, $DRE = E / (E+D)$, where
- E – errors found before shipment
- D – errors found during operation

It provides a strong indication of the effectiveness of the quality assurance activities. Now let us assume that we have collected the following errors and defect data over the last 24 months:

- Errors per requirement specification page – E_{req}
- Error per component – design level – E_{design}
- Errors per component – code level – E_{code}
- DRE – requirement analysis
- DRE – architectural design
- DRE – coding

We now record the number of errors found during each SE step and calculate current values for E_{req} , E_{design} , and E_{code} . These values are then compared to averages of past projects. If the current results vary more than 20% from average, there may be cause for concern and there is certainly cause for investigation.

This can also be used to better target review and/or testing resources in the following manner:

- 120 components

- 32 exhibit $E_{\text{design}} > 1.2$ average
- Adjust code review resources accordingly

Time Boxing

Time-boxing is used in severe deadline pressure. It is a use incremental strategy where tasks associated with each increment are time-boxed in the following manner:

- Schedule for each task is adjusted by working backward from the delivery date.
- A box is put around each task
- When a task hits the boundary of the box, work stops and next task begins.

The principle behind time-boxing is the 90-10 rule (similar to Pareto Principle) rather than becoming stuck on the 10% of a task, the product proceeds towards the delivery date in 90% of the cases.

Lecture No. 27 Software Quality Assurance (SQA)

Quality cannot be assured without first understanding its nature and characteristics.

What is quality?

Software quality is defined as conformance to explicitly stated functional and nonfunctional requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

Emphasizes of quality?

- Software requirements are the foundation from which quality is measured. Lack of conformance is lack of quality.
- Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.
- A set of implicit requirements often goes unmentioned (ease of use, good maintainability etc.)

Literature, quality

In the literature, quality has been defined through in many different manners. One group believes that the quality has measurable characteristic such as cyclomatic complexity, cohesion, and coupling.

We can then talk about quality from different aspects. Quality of design tries to determine the quality of design related documents including requirements, specifications, and design. Quality of conformance looks at the implementation and if it follows the design then the resulting system meets its goals then conformance quality is high.

Glass quality

Glass defines quality as a measure of user satisfaction which is defined by compliant product + good quality + delivery within budget and schedule DeMarco defines product quality as a function of how much it changes the world for the better.

Quality Assurance

Goal of quality assurance is to provide the management with the necessary data to be informed about product quality. It consists of auditing and reporting functions of management. If data provided through QA identifies problems, the management deploys the necessary resources to fix it and hence achieves desired quality control.

Cost of quality

A very significant question is: does quality assurance add any value. That is, is worth spending a lot of money in quality assurance practices? In order to understand the impact of quality assurance practices, we have to understand the cost of quality (or lack thereof) in a system. Quality has a direct and indirect cost in the form of cost of prevention, appraisal, and failure.

This cost includes:

- Quality planning
- Formal technical reviews
- Test equipment
- Training

The cost of appraisal includes activities to gain insight into the product condition. It involves in-process and inter-process inspection and testing. And finally, failure cost. Failure cost has two components:

- internal failure cost
- External failure cost.

Internal failure cost requires rework, repair, and failure mode analysis. On the other hand,

External failure cost involves cost for complaint resolution, product return and replacement, help-line support, warranty work, and law suits. It is trivial to see that cost increases as we go from prevention to detection to internal failure to external failure.

SQA Activities

There are two different groups involved in SQA related activities:

- Software engineers who do the technical work
- SQA group who is responsible for QA planning, oversight, record keeping, analysis, and reporting.

Software engineers address quality

By applying solid technical methods and measures, conducting formal and technical reviews, and performing well planned software testing. The SQA group assists the software team in achieving a high quality product.

SQA Group Activities

An SQA plan is developed for the project during project planning and is reviewed by all stake holders. The plan includes the identification of:

- Evaluations to be performed
- Audits and reviewed to be performed
- Standards that are applicable to the project
- Procedures for error reporting and tracking
- Documents to be produced by the SQA group
- Amount of feedback provided to the software project team

The group participates in the development of the project's software process description. The software team selects the process and SQA group reviews the process description for compliance with the organizational policies, internal software standards, externally imposed standards, and other parts of the software project plan.

The basis purpose is to ensure that deviations in software work and work products are documented and handled according to documented procedures. The group records any non-compliance and reports to senior management and non-compliant items are recorded and tracked until they are resolved. Another very important role of the group is to coordinate the control and management of change and help to collect and analyze software metrics.

Quality Control

The basic principle of quality control is to control the variation as variation control is the heart of quality control. It includes resource and time estimation, test coverage, variation in number of bugs, and variation in support. So quality control is a combination of measurement and feedback and combination of automated tools and manual interaction.

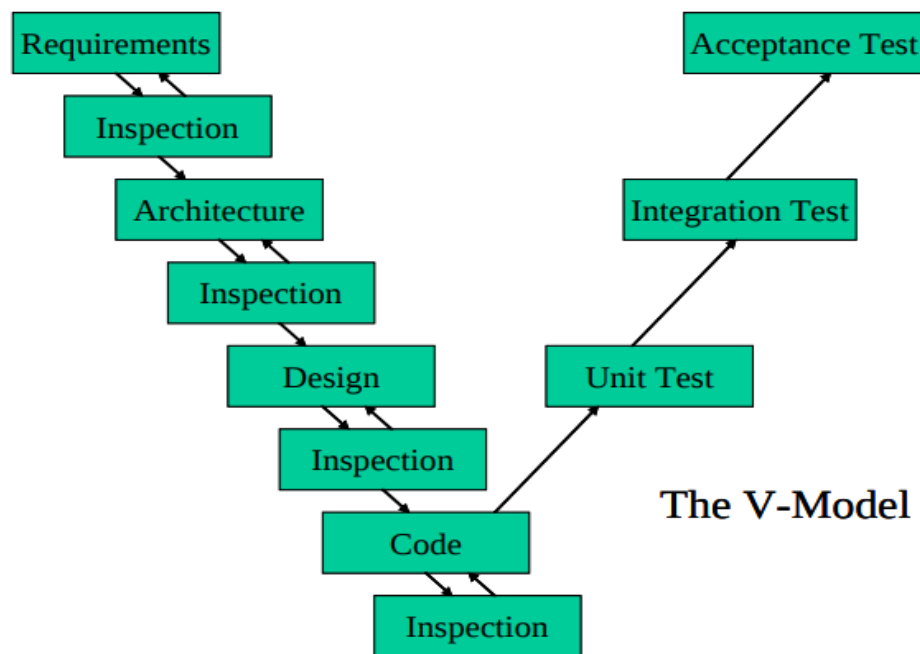
Lecture No. 28 Software Reviews

Software Reviews

Software reviews are the filter for the software engineering process. They're applied at various different points and serve to uncover errors that can be removed and help to purify the software engineering activities.

It is useful to look at the "V-model" of software development. This model emphasizes that SQA is a function performed at all stages of software development life cycle. At the initial stages

(requirement, architecture, design, code), it is achieved through activities known as Formal Technical Reviews or FTR. At the later stages (integration and acceptance).



Importance of reviews by Freedman

“Technical work needs reviewing for the same reason that a pencil needs erasers: To err is human. The second reason that we need technical reviews is although that people are good at catching errors, large class of errors escape the originator more easily than they escape anyone else”.

Freedman says A review – any review – is a way of using the diversity of a group of people to:

- Point out needed improvements in the product of a single person or team
- Confirm those parts of a product in which improvement is either not desired or no needed
- Achieve technical work of more uniform, or at least more predictable, quality than can be achieved without reviews, in order to make technical work more manageable.

Reviews help the development team in improving the defect removal efficiency and hence play an important role in the development of a high-quality product.

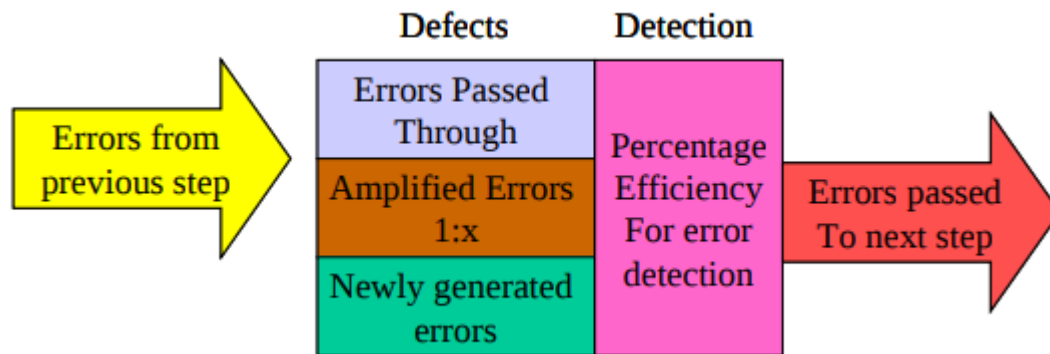
Types of Reviews

There are many types of reviews. In general they can be categorized into two main categories:

- informal
- Formal technical reviews.

Formal Technical reviews are sometimes called as walkthroughs or inspections. They are the most effective filter from QA standpoint.

This model depicts that each development step inherits certain errors from the previous step. Some of these errors are just passed through to the next step while some are worked on and hence are amplified with a ratio of 1:x. In addition, each step may also generate some new errors.



Important note:-

We do not have any SQA related activities for the first two stages and we are only using testing for detection of any defects.

Lecture No. 29 Formal Technical Reviews

Formal Technical Reviews

Formal Technical Reviews are conducted by software engineers. The primary objective is to find errors during the process so that they do not become defects after release of software as they uncover errors in function, logic design, or implementation.

They also ensure that the software has been represented according to predefined standards and it is developed in a uniform manner. They make projects more manageable and help groom new resources as well as provide backup and continuity.

FTRs include walkthroughs, inspections, and other small group technical assessments of software.

Guidelines for walkthroughs

FTRs are usually conducted in a meeting that is successful only if it is properly planned, controlled, and attended. The producer informs the PM that the WP is ready and the review is needed. The review meeting consists of 3-5 people and advanced preparation is required. It is important that this preparation should not require more than 2 hours of work per person. It should focus on specific (and

small) part of the overall software. For example, instead of the entire design, walkthroughs are conducted for each component, or small group of components. By narrowing focus, FTR has a high probability of uncovering errors.

It is important to remember that the focus is on a work product for which the producer of the WP asks the project leader for review. The agenda is also prepared by the review leader.

Review Meetings

Review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes the roles of recorder. Producer walks through the product, explaining the material while other reviewers raise issues based upon their advanced preparation. At the end of the RM, all attendees of the meeting must decide whether to:

- Accept the product without further modification
- Reject the product due to severe errors
 - Major errors identified
 - Must review again after fixing
- Accept the product provisionally
 - Minor errors to be fixed
 - No further review

Review Reporting and Record keeping

During the FTR the recorder notes all the issues. They are summarized at the end and a review issue list is prepared. A summary report is produced that includes:

- What is reviewed
- Who reviewed it
- What were the findings and conclusions

It then becomes part of project historical record.

The review issue list

It is sometimes very useful to have a proper review issue list. It has two objectives.

- Identify problem areas within the WP
- Action item checklist

It is important to establish a follow-up procedure to ensure that items on the issue list have been properly addressed.

Review Guidelines

It is essential to note that an uncontrolled review can be worse than no review. The basis principle is that the review should focus on the product and not the producer so that it does not become personal.

Remember to be sensitive to personal egos. Errors should be pointed out gently and the tone should be loose and constructive. This can be achieved by setting an agenda and maintaining it. In order to do so, the review team should:

- Avoid drift
- Limit debate and rebuttal
- Enunciate problem areas but don't try to solve all problems
- Take written notes
- Limit the number of participants and insist upon advanced preparation
- Develop a checklist for each product that is likely to be reviewed
- Allocate resources and schedule time for FTRs
- Conduct meaningful training for all reviewers
- Review your early reviews
- Determine what approach works best for you

Lecture No. 30 Statistical Software Quality Assurance

Statistical Software Quality Assurance

Statistical SQA is a technique that measures the quality in a quantitative fashion. It implies that information about defects is collected and categorized and an attempt is made to trace each defect to underlying cause. It uses Pareto Principle to identify vital causes (80% of defects can be traced to 20% of causes) and moves to correct the problems that have caused the defects. Example

Let us assume that information about defects is collected for one year and categorized as follows:

1. Incomplete or erroneous specifications (IES)
2. Misinterpretation of customer communication (MCC)
3. Intentional deviation from specifications (IDS)
4. Violation of programming standards (VPS)
5. Error in data representation (EDR)
6. Inconsistent component interface (ICI)
7. Error in digital logic (EDL)
8. Incomplete or erroneous testing (IET)
9. Inaccurate or incomplete documentation (IID)
10. Error in programming language translation of design (PLT)
11. Ambiguous or inconsistent HCI (HCI)
12. Miscellaneous (MIS)

The following data is collected for these categories

Error Category	Serious	Moderate	Minor	Sub Total
IES	34	68	103	205
MCC	12	68	76	156
IDS	1	24	23	48
VPS	0	15	10	25
EDR	26	68	36	130
ICI	9	18	31	58
EDL	14	12	19	45
IET	12	35	48	95
IID	2	20	14	36
PLT	15	19	26	60
HCI	3	17	8	28
MIS	0	15	41	56
Total	128	379	435	942

We now start corrective action focused on vital few. For example for EDR we review the data representation techniques to identify the possible improvement areas and adopt a use case tool for data modeling and perform stringent data design reviews.

Error Index (EI)

Another statistical technique known as Error Index (EI) is used to develop an overall indication of improvement in software quality. The EI is computed as follows:

- E_i – the total number of errors uncovered during the i th step in the SE process
- S_i – number of serious errors
- M_i – number of moderate errors
- T_i – number of minor errors
- PS_i – product size at the i th step
- ws, wm, wt – weighting factors for serious, moderate, and minor errors
- Recommended values – 10, 3, 1 respectively

At each step of the software process a Phase Index is computed

- $Pli = ws(S_i/E_i) + wm(M_i/E_i) + wt(T_i/E_i)$
- EI – cumulative effect on each $Pli = \sum(i \times Pli)/PS_i$

Weighting errors encountered in the SE processes more heavily than those encountered earlier. Used to develop an overall indication of improvement in software quality

Software Reliability

Software reliability is another very important quality factor and is defined as probability of failure free operation of a computer program in a specified environment for a specified time.

Software reliability can be measured, directed, and estimated using historical and development data. The key to this measurement is the meaning of term failure. Failure is defined as non-conformance to software requirements. It can be graded in many different ways as shown below:

- From annoying to catastrophic
- Time to fix from minutes to months
- Ripples from fixing

It is also pertinent to understand the difference between hardware and software reliability. Hardware reliability is predicted on failure due to wear rather than failure due to design. In the case of software, there is no wear and tear. The reliability of software is determined by Mean time between failures (MTBF). MTBF is calculated as:

$$MTBF = MTTF + MTTR$$

Where MTTF is the Mean Time to Failure and MTTR is the Mean time required to Repair.

Where MTTF is the Mean Time to Failure and MTTR is the Mean time required to Repair. Arguably MTBF is far better than defects/kloc as each error does not have the same failure rate and the user is concerned with failure and not with total error count. A related issue is the notion of availability. It is defined as the probability that a program is operating according to requirements at a given point in time. It can be calculated as

$$\text{Availability} = (MTTF/MTBF) \times 100$$

and clearly depends upon MTTR.

Lecture No. 31 Software Safety

Software Safety

Software Safety is a software SQA activity that focuses on identification of potential hazards that may affect software negatively and cause an entire system to fail. Modeling and analysis process is conducted as part of software safety and hazards are identified and categorized by criticality and risk.

Example

Let us assume that the following hazards are associated with a computer-based cruise control for an automobile:

- Causes uncontrolled acceleration that cannot be stopped
- Does not respond to depression of brake pedal
- Does not engage when switch is activated
- Slowly loses or gains speed

Once system-level hazards are identified, analysis techniques are used to assign severity, and probability of occurrence. This technique is similar to risk analysis. To be effective, software must be analyzed in the context of the entire system.

Analysis techniques

Analysis techniques such as fault tree analysis can be used to predict the chain of events that can cause hazards and the probability that each of these events will occur to create the chain. Once hazards are identified and analyzed, safety-related requirements can be specified for the software.

Reliability and safety are closely related:

Software reliability uses statistical techniques to determine the likelihood that a software failure will occur. Occurrence of a software failure does not necessarily result in a hazard or mishap. On the other hand, software safety examines the ways in which failures result in conditions that can lead to a mishap.

Lecture No. 32 Poka-Yoke (Mistake-Proofing)

Poka-Yoke (Mistake-Proofing)

Poka-yoke is a QA technique developed by Shingo at Toyota in 1960's. Poka-yoke devices are mechanisms that lead to prevention of potential quality problem before it occurs or the rapid detection of quality problems if they are introduced.

Characteristic of a Poka-yoke device

- It is simple and cheap
- It is part of the process
- It is located near the process task where the mistake occurs

Example of Poka-yoke in software

Let us assume that a software Products Company sells application software to an international market. The pull-down menus and associated mnemonics provided with each application must reflect the local language. For example, the English language menu item for "Close" has the mnemonic "C" associated with it. When the application is sold to Urdu speaking users, the same menu item is "Band Karen" with mnemonic "ب". To implement the appropriate menu entry for each locale, a "localizer" translates the menus accordingly. The problem is to ensure that each menu entry conforms to appropriate standards and there are no conflicts, regardless of the language used.

Lecture No. 33 Software Configuration Management (SCM)

LEVEL 1 - INITIAL

LEVEL 2 -REPEATABLE

- REQUIREMENTS MANAGEMENT

- PROJECT PLANNING
- PROJECT TRACKING & OVERSIGHT
- SUBCONTRACT MANAGEMENT
- SOFTWARE QUALITY ASSURANCE
- SOFTWARE CONFIGURATION MANAGEMENT

Software Configuration Management (SCM)

Software configuration management (SCM) is one of the five KPA required for an organization to be at CMM level 2. That means, according to SEI, effective project management is not possible without having a proper SCM function in place.

The “First Law” of Change

No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle.

SCM - CMM

“To establish and maintain the integrity of software products throughout the project’s life cycle” The basic idea behind SCM is to manage and control change. As mentioned by Bersoff, no matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle. We are getting more and more complex software projects in terms of size, sophistication, and technology. In addition, these software systems are used by millions of users all over the world. These systems need multilingual and multi-platform support (hardware, software) and have to operate in a distributed environment. That means that a software system may come in many configuration flavors including desktop, standard, professional, and enterprise versions.

Change Chaos

This frequent change, if not managed properly, results in chaos. First of all there would be problems of identification and tracking which would result in questions like the following:

- “This program worked yesterday. What happened?”
- “I fixed this error last week. Why is it back?”
- “Where are all my changes from last week?”
- “This seems like an obvious fix. Has it been tried before?”
- “Who is responsible for this change?”

SCM is a function that, if implemented, will reduce these problems to a minimal level.

Configuration management

As defined by CMM, the purpose of SCM is to establish and maintain the integrity of software products through the project’s life cycle. Configuration management is concerned with managing evolving software systems. It acknowledges that system change is a team activity and thus it aims to control the

costs and effort involved in making changes to a system. SCM involves the development and application of procedures and standards to manage an evolving software product and is part of a more general quality management process.

Baseline

When released to CM, software systems are called baselines and serve as the starting point for further development. A baseline is a software configuration management concept that helps us to control change without seriously impeding justifiable change. It is defined by IEEE as:

“A specification or a product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures.”

Software Configuration Item (SCI)

A Software Configuration Item is the information that is created as part of the software engineering process. Typical SCIs include requirement specifications, design specification, source code, test cases and recorded results, user guides and installation manuals, executable programs, and standards and procedures (for example C++ design guidelines).

Software Configuration Management Tasks

Software configuration management tasks include:

- Identification
- Version Control
- Change Control
- Configuration Auditing
- Reporting

Identification

How does an organization identify and manage the many existing versions of a program in a manner that will enable changes to be accommodated efficiently?

- Identify baseline
- Identify configurable software items
- At the minimum, all deliverables must be identified as configurable items- design, software, test cases, tutorials, user guides...

Version Control

How does an organization control changes before and after software is released to a customer?

Clemm states that

Configuration management allows the user to specify alternative configurations of the software system through the selection of the appropriate versions. This is supported by associating with each software version, and then allowing configuration to be specified and constructed by describing the set of desired attributes.

A version has many different attributes. In the simplest form a specific version number that is attached to each object and in the complex form it may have a string of Boolean variables (switches) that indicate specific types of functional changes that have been applied to the system.

Change Control

Who has the responsibility for approving and ranking changes?

The Change Control process addresses the important question of who has the responsibility for approving and ranking changes. Configuration Identification involves identification of a tool for SCM. Then a baseline is established and identified which is then used to identify configurable software items. At the minimum, all deliverables must be identified as configurable items. This includes design, software, test cases, tutorials, and user guides.

Configuration Auditing

How can we ensure that the changes have been made properly?

Reporting

What mechanism is used to appraise others of changes that are made?

Lecture No. 34 Product Release Version Numbering System

Product Release Version Numbering System

Product release is the act of making a product available to its intended customers. After a product has had its first release, it enters a product release cycle. New versions of the product are made available that may fix defects or add features that were not in previous releases. These changes are categorized as updates or upgrades. An update fixes product defects. An upgrade enhances the product feature set and will include updates.

Release Numbering

Each individual product release is viewed as being in a unique state which is the total set of functionality possessed by the product release. Although there is no industry standard, typically, a three field compound number of the format “X.Y.Z” is used.

The first digit, X, is used for the major release number which is used to identify a major increase in the product functionality. The major release number is usually incremented to indicate a significant change in the product functionality or a new product base-line.

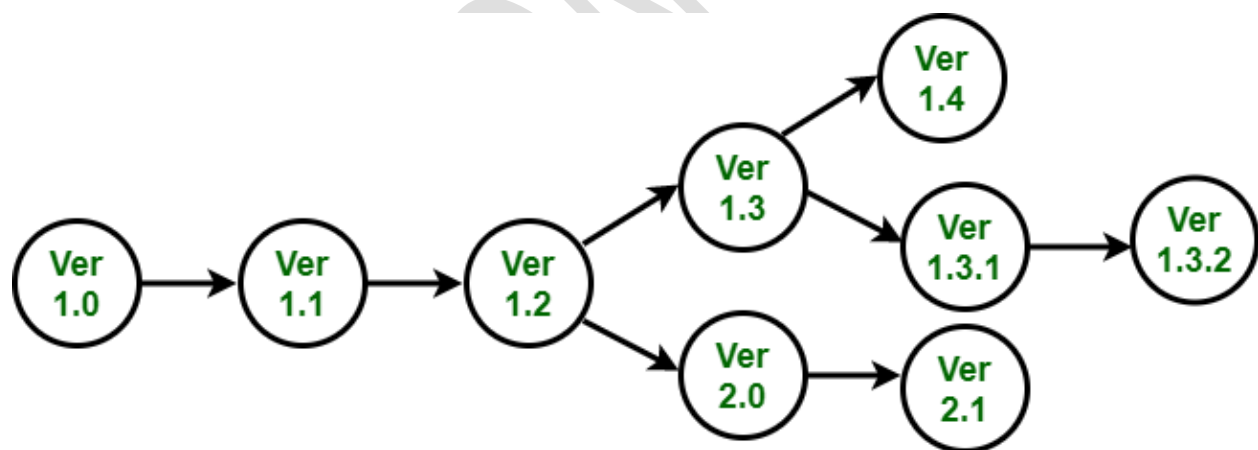
The second digit, Y, stands for feature release number. The feature release number is iterated to identify when a set of product features have been added or significantly modified from their originally documented behaviour.

The third digit, Z, is called the defect repair number and is incremented when a set of defects is repaired. Defect repair/maintenance is considered to be any activity that supports the release functionality specification and it may a fix for some bugs or some maintenance to enhance the performance of the application.

Conventionally, a release number starts with a major number of one, followed by zero for its feature and maintenance numbers. This results in a release number 1.0.0. If the first new release that is needed is a defect repair release, the last digit would be iterated to one, resulting in 1.0.1. If two additional defect repair releases are needed, we would eventually have a release number of 1.0.3.

Internal Release Numbering

A special type of release is internal release. Internal releases are used by the development organization as a staging mechanism for functionality. The most common internal releases are the regular builds. It is suggested that we add an extra (fourth) field to identify and keep track of internal builds.



Change control

James Back points out the difficulties related to change control as follows:

Change control is vital. But the forces that make it necessary also make it annoying. We worry about change because a tiny perturbation in the code can cause a big failure in the product. But it can also fix a big failure or enable wonderful new capabilities. We worry about change because a single rogue developer could sink the project; yet brilliant ideas originate in the minds of those rogues, and a burdensome change control process could discourage them from doing creative work.

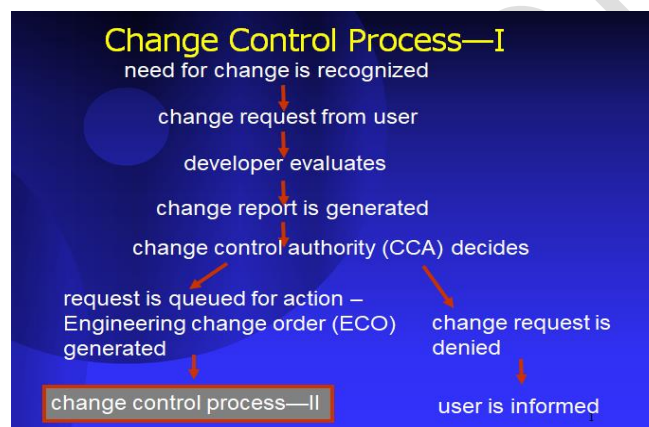
Balancing act

- Too much or too little change control creates different types of problems
 - Uncontrolled change rapidly leads to chaos
 - Too much control leads to frustration.
 - Change control mechanism - procedures and tools

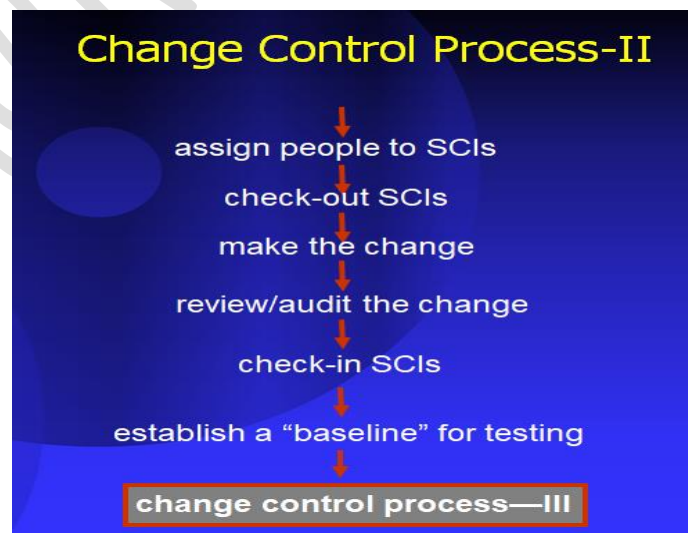
Lecture No. 35 Change Control Process

Change Control Process

The first component of the change control process is the Change Control Authority (CCA) or a Change Control Board (CCB). A CCA or CCB includes people from both developer and client side.

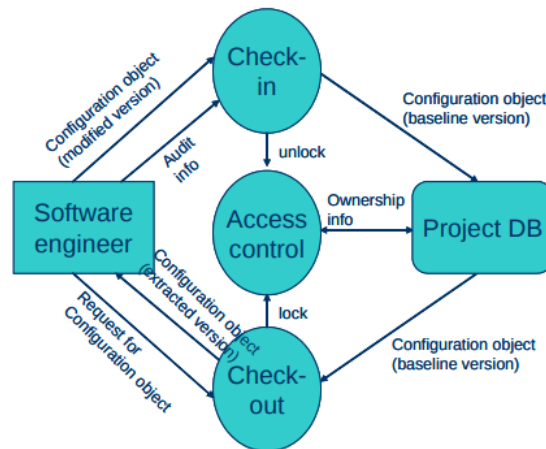
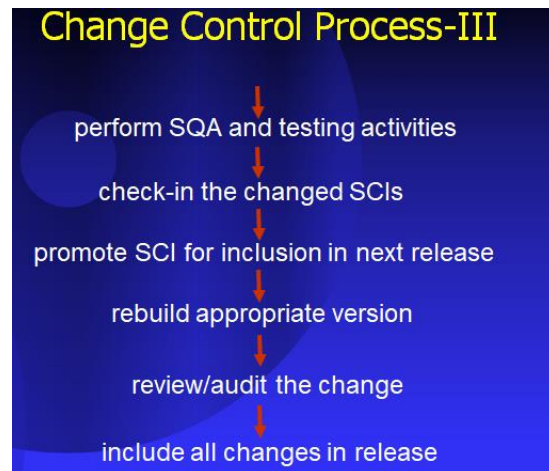


Whenever a change is required, the CCB decides whether to allow this change to happen or deny it. If it is decided that a change is needed, an Engineering Change Order or ECO is generated. An ECO defines the change to be made, the constraints that must be respected, and the criteria for review and audit. The change control process thus involves the following steps.



Check-in and check-out

Access and synchronization control who has the authority to check-out the object. Synchronization control ensures that parallel changes by two different people do not overwrite one another. Control on updates, other copies can be checked out for use, Single writer, and multiple readers.



Configuration Audit

Configuration audit ensures that a change has been properly implemented. It involves formal technical reviews and software configuration audit. Configuration audit assess a configuration object for characteristics that are generally not considered during audit.

Configuration Status reporting (CSR)

Configuration Status Reporting (CSR) is also known as status accounting. It reports on the following items:

- What happened?
- Who did it?
- When did it happen?
- What else will be affected?

If it is not done then the organization faces the left hand not knowing what the right hand is doing syndrome. CSR reports are generated on regular basis. Each time an SCI is assigned a new identification, a CSR entry is made. Each time a change is approved by CCA, a CSR entry is made.

Lecture No. 36 Requirement Management and CMM

Requirement Management and CMM

CM should always be based on a set of standards which are applied within an organisation. Standards should define how items are identified; how changes are controlled and how new versions are managed. Standards may be based on external CM standards (e.g. IEEE standard for CM ANSI/IEEE Std. No. 828-

1983, 1042-1987, 1028- 1988). Existing standards are based on a waterfall process model - new standards are needed for evolutionary development.

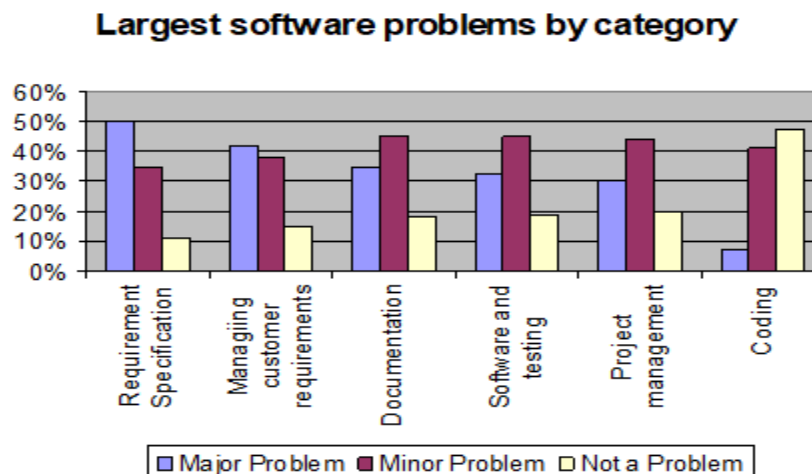
The Requirement Problem

The goal of software development is to develop quality software on time and on budget that meets customers' real needs. Project success depends on good requirement management. Requirement errors are the most common type of software development errors and the most costly to fix.

The root causes of project failure (The Standish Group)

- Lack of user input is responsible for : 13% of all project failures
- Incomplete requirements and specifications: 12% of all project failures
- Changing requirements: 12% of all project failures

European Software Process Improvement Training Institute Survey for Largest Software problems



Requirement Management

A systematic approach to eliciting, organizing, and documenting the requirements of the system, and a process that establishes and maintains agreement between the customer and the project team on the changing requirements of the system.

Establishing and maintaining an agreement with the customer on the requirement for the software project. It involves

- Defining the requirement baseline
- Reviewing proposed requirement changes and evaluating the likely impact of each proposed change before deciding whether to approve it
- Incorporating approved requirement changes in the project in a controlled manner.

- Keeping project plans current with the requirements
- Negotiating new commitments based on estimated impact on changed requirements
- Tracing individual requirements to their corresponding design, source code, and test cases
- Tracking requirement status and change activity throughout the project

Requirement Attributes

Go beyond the description of intended functionality. Attributes are used to establish a context and background for each requirement. Can be used to filter, sort, or query to view selected subset of the requirements.

Attributes

- Requirement ID
- Creation date
- Created by
- Last modified on
- Last modified by
- Version number
- Status
- Origin
- Subsystem
- Product Release
- Priority

Requirement Status

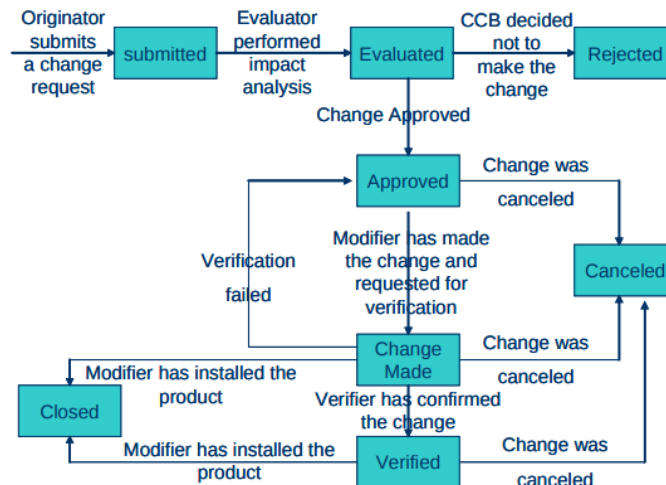
The requirement status attribute is one of the most useful ones. It can be used to keep track of different requirements going through different phases. The possible status values are proposed, approved, implemented, verified, and deleted.

1. **Proposed:**
The requirement has been requested by a source that has the authority to provide requirements.
2. **Approved:**
The requirement has been analyzed, its impact on the rest of the project has been estimated, and it has been allocated to the baseline for a specific build number or product release. The software development group has committed to implement the requirement.
3. **Implemented:**
The code that implements the requirement has been designed, written, and unit tested.
4. **Verified:**
The implemented requirement has been verified through the selected approach, such as testing or inspection. The requirement has been traced to pertinent test cases. The requirement is now considered complete.
5. **Deleted:**

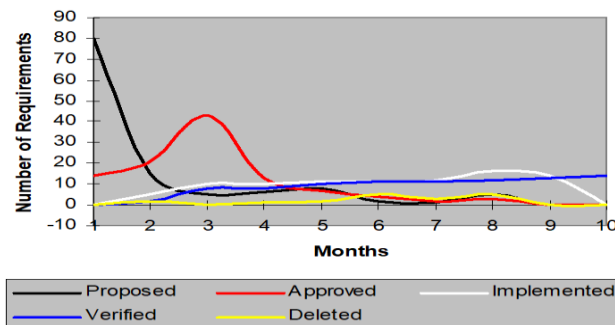
A planned requirement has been deleted from the baseline. Include an explanation of why and by whom the decision was made to delete the requirement.

Change Request Status

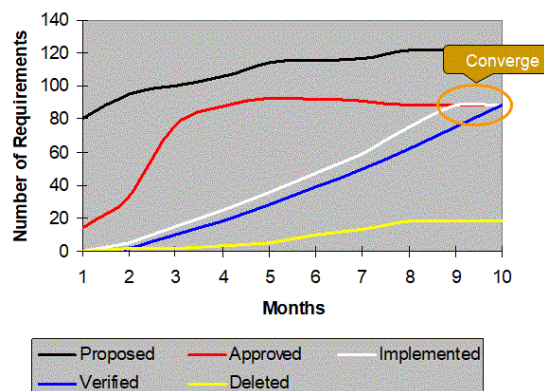
As the requirement go through different phases, their status is updated accordingly. The following state transition diagram captures the sequence of requirement status changes.



Requirement Status Chart



Requirement Status Chart



Managing Scope Creep

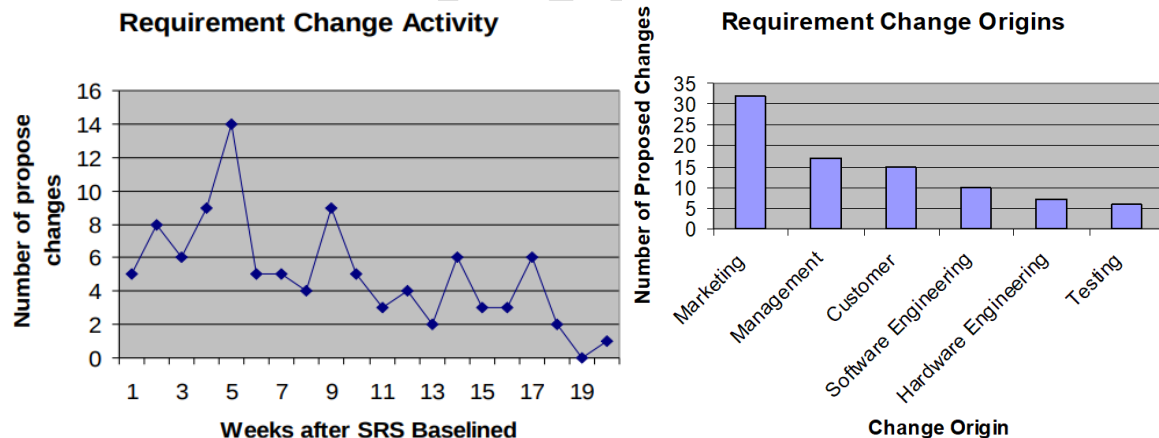
We must always remember that requirements will change, no matter what. That means we have to be able to manage changing requirements. Software organizations and professionals must learn to manage changing requirements. A major issue in requirements engineering is the rate at which requirements change once the requirements phase has “officially” ended. We therefore need to try to take it to a minimum level. For that we need to measure the change activity.

Lecture No. 37 Measuring Change Activity

Measuring Change Activity

Measuring Change Activity Measurement of change activity is a way to assess the stability of the requirements and to identify opportunities for process improvement. In this regards, the following could be measured

- The number of change requests received, open, and closed
- The cumulative number of changes made including added, deleted, and modified requirements
- The number of change requests that originated from each source
- The number of changes proposed and made in each requirement since it was baselined
- The total effort devoted to handling changes



Requirement Traceability

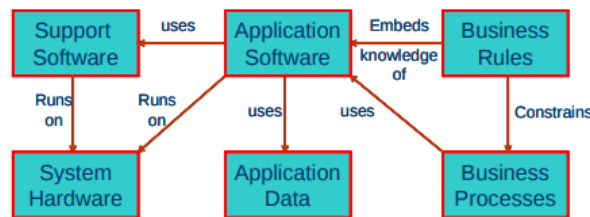
Requirement traceability is a very important consideration for requirement management. It is really hard to manage requirements that are not traceable.

A Software Requirement Specification (SRS) is traced if the origin of its requirements is clear. That means that the SRS includes references to earlier supportive documents. An SRS is traceable if it is written in a manner that facilitates the referencing of each individual requirement stated therein.

Lecture No. 38 Legacy systems

Legacy systems

A system is considered to be a legacy system if it has been in operation for many years. A legacy system has many components. These include business processes, business rules, application software, application data, support software, and system hardware.



Maintaining Legacy System

Maintaining legacy system is expensive. Different parts of the system have been implemented by different teams, lacking consistency. Part or all of the system may be implemented using an obsolete language. System documentation is inadequate and out of date.

- In some cases the only documentation is the source code.
- In some cases even the source code is not available.

Many years of maintenance have usually corrupted the system structure, making it increasingly difficult to understand. The data processed by the system may be maintained in different files which have incompatible structures. There may be data duplication and the documentation of the data itself may be out of date, inaccurate, and incomplete.

System Hardware

The hardware platform may be outdated and is hard to maintain. In many cases, the legacy systems have been written for mainframe hardware which is no longer available, expensive to maintain, and not be compatible with current organizational IT purchasing policies.

Support software

- OS, database, and compiler etc.
- May be obsolete and no longer supported by the vendors.

Legacy migration risks

There is rarely a complete specification of the system available. Therefore, there is no straight forward way of specifying the services provided by a legacy system. Important business rules are often embedded in the software and may not be documented elsewhere. Business processes and the way legacy systems operate are often intertwined. New software development may take several years. New software development is itself risky. Changes to one part of the system inevitably involve further changes to other components.

Legacy System Assessment

For each legacy system, there are four strategic options:

- Scrap the system completely
- Continue maintaining the system
- Transform the system in some way to improve its maintainability
- Replace the system with a new system

Scrap the system completely

System is not making an effective contribution to business processes. Business processes have changed significantly and the organization is no longer completely dependent upon the system.

Continue maintaining the system

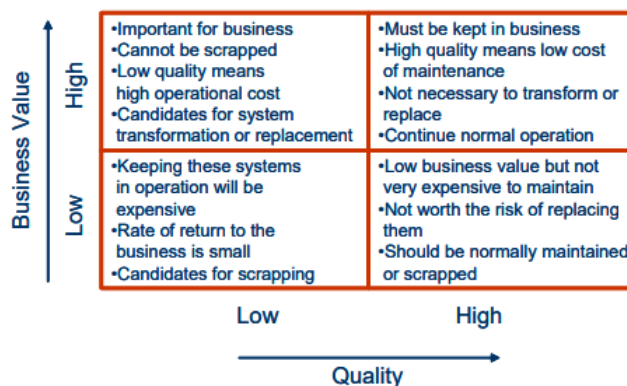
System is still required, it is stable, and Requirements are not changing frequently.

Transform the system in some way to improve its maintainability

System quality has been degraded. Regular changes to the system are required.

Replace the system with a new system

Old system cannot continue in operation. Off-the shelf alternative is available or system can be developed at a reasonable cost.



Business Value Assessment

It is important to note that this is a subjective judgment and requires different business viewpoints. These viewpoints include end-users, customers, line managers, IT managers, and senior managers.

End Users

- How effective do they find the system in supporting their business processes
- How much of the system functionality is used.

Customers

- Is the use of the system transparent to customer or are their interaction constrained by the system
- Are they kept waiting because of the system
- Do system errors have a direct impact on the customer?

IT Managers

- Are there difficulties in finding people to work on the system?
- Does the system consume resources which could be deployed more effectively on other systems?

Line Managers

- Do managers think that the system is effective in contributing to success of their unit?
- Is the cost of keeping the system in use justified?
- Is the data managed by the system critical for the functioning of the manager's unit?

Senior Managers

- Does the system and associated business process make an effective contribution to the business goal?

Lecture No. 39 Environment Assessment

Environment Assessment

The legacy system also needs to be assessed from an environment's perspective. This involves looking at the supplier, failure rate, age, performance, support requirements, maintenance cost, and interoperability.

Application software assessment

The application software is assessed on the following parameters:

Understandability

How difficult is it to understand the software code of the current system. How complex are the control structures that are used.

Documentation

What system documentation is available? Is the documentation complete, consistent, and up-to-date?

Data

Is there an explicit data model for the system? To what extent is data duplicated in different files.

Programming Language:

Are modern compilers available for the programming language? Is the language still used for new system development?

Test Data:

Does test data for the system exist? Is there a record of regression tests carried out when new features have been added to the system?

Personnel skills:

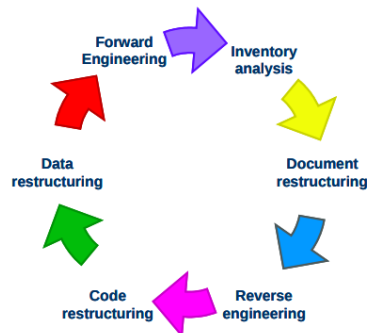
Are there people available who have the skills to maintain the system?

Software Reengineering

Software solutions often automate the business by implementing business rules and business processes. A time comes when these changes become very difficult to handle. So reengineering is re-implementing legacy systems to make them more maintainable. It is a long term activity.

Software Reengineering Process Model

The software reengineering is a non-trivial activity. Just like legacy migration, careful analysis must be carried out before a decision for reengineering is taken. The following process model can be used to reengineer a legacy system.



Inventory analysis

Inventory analysis is the first step in the reengineering process. At this stage, inventory of all applications is taken a note of their size, age, business criticality, and current maintainability is made.

Document restructuring

The next step in the reengineering process is document restructuring. Weak documentation is a trademark of many legacy applications. Without proper documentation, the hidden rules, business processes, and data cannot be easily understood and reengineered.

In this regards, the following options are available:

1. Create documentation:

Creating documentation from scratch is very time consuming. If program is relatively stable and is coming to the end of its useful life then just leave it as it is.

2. Update documentation:

This option also needs a lot of resources. A good approach would be to update documentation when the code is modified.

Reverse engineering

Reverse engineering is the next step in the process. Reverse engineering for software is a process for analyzing a program in an effort to create a representation of the program at a higher level of abstraction than the source code. Reverse engineering is the process of design recovery. At this stage, documentation of the overall functionality of the system that is not there is created.

Reverse engineering activities include:

- Reverse engineering to understand processing
- Reverse engineering to understand data, internal data structure, Database structures.
- Reverse engineering user interfaces.

Program Restructuring

Program is restructured after the reverse engineering phase. In this case we modify source code and data in order to make it amenable to future changes. This includes code as well as data restructuring.

Lecture No. 40 Forward Engineering

Forward Engineering

Application of SE principles, methods, and concepts to re-create an existing application In most cases forward engineering does not simply create a modern equivalent of an older program, rather new user and technology requirements are integrated into the reengineering effort.

The Economics of Reengineering

Cost benefit analysis

- P1 : current annual maintenance cost for an application
- P2 : current annual operation cost for an application
- P3 : current annual business value of an application
- P4 : predicted annual maintenance cost after reengineering
- P5 : predicted annual operations cost after reengineering
- P6 : predicted annual business value cost after reengineering
- P7 : estimated reengineering cost
- P8 : estimated reengineering calendar time
- P9 : reengineering risk factor (1.0 is nominal)
- L : expected life of the system

Now the cost of maintenance is calculated as:

$$C_{\text{maintenance}} = [P3 - (P1 + P2)] \times L$$

Cost of reengineering would then be given by the formula:

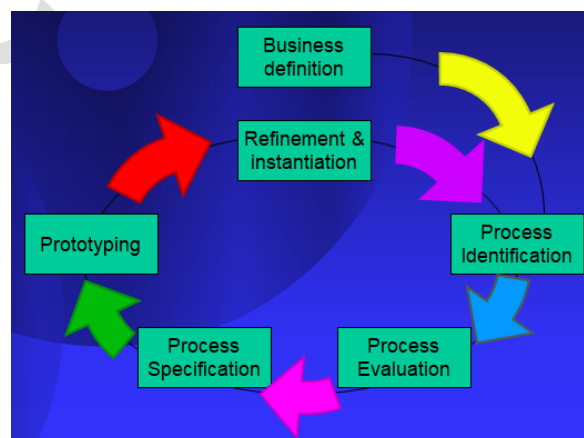
$$C_{\text{reengineering}} = [P6 - (P4 + P5) \times (L - P8) - (P7 \times P9)]$$

Lecture No. 41 Business Process Reengineering

Business Process Reengineering

The search for, and implementation of, radical change in business process to achieve breakthrough results (Fortune Magazine). A concept similar to software reengineering is of business process reengineering (BPR). A business process is “a set of logically related tasks performed to achieve a defined business outcome”.

For BPR the following process model may be used.



Principles of BPR

- Organize around outcomes, not tasks
- Have those who use the output of the process perform the process
- Incorporate information processing work into the real work that produces the raw information
- Treat geographically dispersed resources as though they were centralized
- Link parallel activities instead of integrating their results
- Put the decision point where the work is performed, and build control into the process
- Capture data once, at its source.

Philosophy

Refactoring is the process of changing a software system such that:

- The external behavior of the system does not change
- The internal structure of the system is improved
- This is sometimes called “Improving the design after it has been written”.

Fowler’s definition

Refactoring (noun)

A change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.

Refactoring (verb)

To restructure software by applying a series of refactoring without changing its observable behavior.

Lecture No. 42 Software Refactoring

Software Refactoring

Software refactoring is the process of changing a software system such that the external behavior of the system does not change while the internal structure of the system is improved. This is sometimes called “Improving the design after it has been written”.

A (Very) Simple Example

Let us consider a very simple example. The refactoring involved in this case is known as “Consolidate Duplicate Conditional Fragments”.

A (Very) Simple Example

```
if (isSpecialDeal()) {
```

```
total = price * 0.95;
```

```
send ();
```

```
}
```

```
else {
```

```
total = price * 0.98;
```

```
send ();
```

```
}
```

In this case, `send` is being called from different places. We can consolidate as follows:

```
if (isSpecialDeal())
```

```
total = price * 0.95;
```

```
else
```

```
total = price * 0.98;
```

```
send ();
```

It can further be improved if we calculate `total` outside the `if` statement as shown below.

```
if (isSpecialDeal())
```

```
factor = 0.95;
```

```
else
```

```
factor = 0.98;
```

```
total = price * factor;
```

```
send ();
```

Refactoring: Where to Start?

Fowler et al has devised a heuristic based approach to this end known as “Bad Smells” in Code. The philosophy is simple: “if it stinks, change it”.

Bad Smells in Code

They have identified many different types of “bad smells”. These are briefly described in the following paragraphs:

- **Duplicated Code** – bad because if you modify one instance of duplicated code but not the others, you (may) have introduced a bug!
- **Long Method** – long methods are more difficult to understand; performance concerns with respect to lots of short methods are largely obsolete
- **Large Class** – Large classes try to do too much, which reduces cohesion
- **Long Parameter List** – hard to understand, can become inconsistent
- **Divergent Change** – Deals with cohesion; symptom: one type of change requires changing one subset of methods; another type of change requires changing another subset
- **Shotgun Surgery** – a change requires lots of little changes in a lot of different classes
- **Feature Envy** – A method requires lots of information from some other class (move it closer!)
- **Data Clumps** – attributes that clump together but are not part of the same class
- **Primitive Obsession** – characterized by a reluctance to use classes instead of primitive data types
- **Switch Statements** – Switch statements are often duplicated in code; they can typically be replaced by use of polymorphism (let OO do your selection for you!)
- **Parallel Inheritance Hierarchies** – Similar to Shotgun Surgery; each time I add a subclass to one hierarchy, I need to do it for all related hierarchies
- **Lazy Class** – A class that no longer “pays its way”. e.g. may be a class that was downsized by refactoring, or represented planned functionality that did not pan out
- **Speculative Generality** – “Oh I think we need the ability to do this kind of thing someday”
- **Temporary Field** -An attribute of an object is only set in certain circumstances; but an object should need all of its attributes
- **Message Chains** – a client asks an object for another object and then asks that object for another object etc. Bad because client depends on the structure of the navigation
- **Middle Man** – If a class is delegating more than half of its responsibilities to another class, do you really need it?
- **Inappropriate Intimacy** – Pairs of classes that know too much about each other’s private details
- **Alternative Classes with Different Interfaces** – Symptom: Two or more methods do the same thing but have different signature for what they do
- **Incomplete Library Class** – A framework class doesn’t do everything you need
- **Data Class** – These are classes that have fields, getting and setting methods for the fields, and nothing else; they are data holders, but objects should be about data AND behavior
- **Refused Bequest** – A subclass ignores most of the functionality provided by its superclass
- **Comments (!)** – Comments are sometimes used to hide bad code – “...comments often are used as a deodorant” (!)

Breaking a Method

Now look at another simple example of long method. Although, in this case, the code is not really long, it however demonstrates how longer segments of code can be broken into smaller and more manageable (may be more reusable as well) code segments.

```
for (i=0; i < N-1; i++) {
```

```

min = i;

for (j = i; j < N; j++)

if (a[j] < a[min]) min = j;

temp = a[i];

a[i] = a[min];

a[min] = temp;

}

```

Extracting the Amount Calculation

The obvious first target of my attention is the overly long statement `()` method. When I look at a long method like that, I am looking to take a chunk of the code and extract a method from it.

Lecture No. 43 Moving the amount calculation

Moving the amount calculation

This method is thus on the wrong object, it should be moved to the rental. To move a method you first copy the code over to rental, adjust it to fit in its new home and compile.

```

Class Rental
double charge() {
double result = 0;
switch (tape().movie().priceCode()) {
case Movie.REGULAR:
result += 2;
if (daysRented() > 2)
result += (daysRented() - 2) * 1.5;
break;
case Movie.NEW_RELEASE:
result += daysRented() * 3;
break;
case Movie.CHILDRENS:
result += 1.5;
if (daysRented() > 3)
result += (daysRented() - 3) * 1.5;
break;
}
return result;
}

```

In this case fitting into its new home means removing the parameter.

Removing Temps

Temporary variables can be a problem. They are only useful within their own routine, and thus they encourage long complex routines. In this case we have two temporary variables, both of which are being used to get a total from the rentals attached to the customer. Both the ascii and html versions will require these totals. I like to replace temps with queries. Queries are accessible to any method in the class, and thus encourage a cleaner design without long complex methods.

I began by replacing totalAmount with a charge() method on customer.

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints = 0;
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + name() + "\n";
    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();
        // add frequent renter points
        frequentRenterPoints += each.frequentRenterPoints();
        //show figures for this rental
        result += "\t" + each.tape().movie().name() + "\t" + String.valueOf(each.charge())
        + "\n";
    }
    //add footer lines
    result += "Amount owed is " + String.valueOf(charge()) + "\n";
    result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter
    points";
    return result;
}

private double charge(){
    double result = 0;
    Enumeration rentals = _rentals.elements();
    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();
        result += each.charge();
    }
    return result;
}
```

After compiling and testing that refactoring, I then did the same for frequentRenterPoints.

```
public String statement() {
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + name() + "\n";
    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();
        //show figures for each rental
        result += "\t" + each.tape().movie().name() + "\t" +
        String.valueOf(each.charge()) + "\n";
    }
}
```

```
//add footer lines
result += "Amount owed is " + String.valueOf(charge()) + "\n";
result += "You earned " + String.valueOf(frequentRenterPoints()) +
" frequent renter points";
return result;
}
private int frequentRenterPoints() {
int result = 0;
Enumeration rentals = _rentals.elements();
while (rentals.hasMoreElements()) {Rental each = (Rental) rentals.nextElement();
result += each.frequentRenterPoints();
}
return result;
}
```

Lecture No. 44 Capability Maturity Model Integration (CMMI)

Capability Maturity Model Integration (CMMI)

Capability Maturity Model or CMM is a reference model of mature practices in a specified discipline, used to assess a group's capability to perform that discipline. In fact there are a number of CMMs. They differ by discipline (software, systems, acquisition, etc.), structure (staged versus continuous), how maturity is defined (process improvement path), and how capability is defined (institutionalization).

Software CMM, the one we saw earlier on, is one of the many CMM models.

Name	Structure	Domain
Software CMM	staged	software development
System Engineering CMM	continuous	system engineering
Software Acquisition CMM	staged	software acquisition
System Security Engineering CMM	continuous	security engineering
FAA-iCMM	continuous	software engineering, systems engineering, and acquisition
IPD-CMM	hybrid	integrated product development
People CMM	staged	workforce
SPICE Model	continuous	software development

CMMI

Integrates systems and software disciplines into single process improvement framework and provides a framework for introducing new disciplines as needs arise.

CMMI Representations

A representation allows an organization to pursue different improvement objectives. There are two types of representations in the CMMI models: staged and continuous.

Staged Representation

Staged representation is classical representation we have already seen previously. It:

- Provides a proven sequence of improvements, each serving as a foundation for the next.
- Permits comparisons across and among organizations by the use of maturity levels.
- Provides an easy migration from the SW-CMM to CMMI.
- Provides a single rating that summarizes appraisal results and allows comparisons among organizations.

Continuous Representation

Continuous representation allows you to select the order of improvement that best meets your organization's business objectives and mitigates your organization's areas of risk. It enables comparisons across and among organizations on a process-area-by-process-area basis and provides an easy migration from EIA 731 (and other models with a continuous representation) to CMMI.

Capability Levels

A capability level is a well-defined evolutionary plateau describing the organization's capability relative to a process area. There are six capability levels. For capability levels 1-5, there is an associated generic goal. Each level is a layer in the foundation for continuous process improvement. Thus, capability levels are cumulative, i.e., a higher capability level includes the attributes of the lower levels. The five (actually six) capability levels (starting from 0) are enumerated below in the reverse order, 5 being the highest and 0 being the lowest.

5 Optimizing

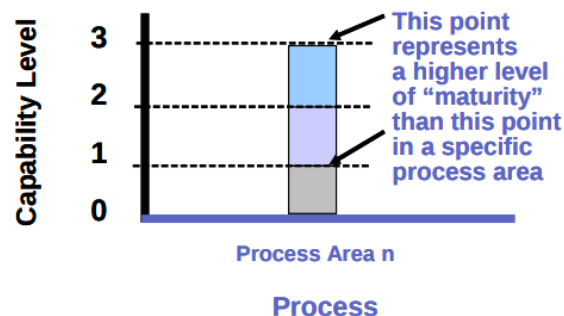
4 Quantitatively Managed

3 Defined

2 Managed

1 Performed

0 Incomplete



Relating Process Area Capability and Organizational Maturity

Organizational maturity and process area capability are similar concepts. The difference between them is that organizational maturity pertains to a set of process areas across an organization, while process area capability deals with a set of processes relating to a single process area or specific practice.

Comparison of Representations

Staged

- Process improvement is measured using maturity levels.
- Maturity level is the degree of process improvement across a predefined set of process areas.
- Organizational maturity pertains to the “maturity” of a set of processes across an organization.

Continuous

- Process improvement is measured using capability levels.
- Capability level is the achievement of process improvement within an individual process area.
- Process area capability pertains to the “maturity” of a particular process across an organization.

Advantages of Each Representation

Staged provides a roadmap for implementing groups of process areas and sequencing of implementation. It has a familiar structure for those transitioning from the Software CMM.

Continuous provides maximum flexibility for focusing on specific process areas according to business goals and objectives and has a familiar structure for those transitioning from EIA 731.

Lecture No. 45 CMM Overview

CMM Maturity Levels

There are five levels defined along the continuum of the CMM and, according to the SEI: "Predictability, effectiveness, and control of an organization's software processes are believed to improve as the organization moves up these five levels. While not rigorous, the empirical evidence to date supports this belief."

Level 1 - Ad hoc (Chaotic)

It is characteristic of processes at this level that they are (typically) undocumented and in a state of dynamic change, tending to be driven in an ad hoc, uncontrolled and reactive manner by users or events. This provides a chaotic or unstable environment for the processes.

Organizational implications

- a) Institutional knowledge tends to be scattered (there being limited structured approach to knowledge management) in such environments, not all of the stakeholders or

participants in the processes may know or understand all of the components that make up the processes.

- b) Despite the chaos, such organizations manage to produce products and services.
- c) Due to the lack of structure and formality, organizations at this level may overcommit, or abandon processes during a crisis, and it is unlikely that they will be able to repeat past successes. There tends to be limited planning, limited executive commitment or buy-in to projects, and limited acceptance of processes.

Level 2 - Repeatable

It is characteristic of processes at this level that some processes are repeatable, possibly with consistent results. Process discipline is unlikely to be rigorous, but where it exists it may help to ensure that existing processes are maintained during times of stress.

Organizational implications

Processes and their outputs could be visible to management at defined points, but results may not always be consistent. For example, for project/program management processes, even though (say) some basic processes are established to track cost, schedule, and functionality, and if a degree of process discipline is in place to repeat earlier successes on projects with similar applications and scope, there could still be a significant risk of exceeding cost and time estimates.

Level 3 - Defined

It is characteristic of processes at this level that there are sets of defined and documented standard processes established and subject to some degree of improvement over time. These standard processes are in place (i.e., they are the AS-IS processes) and used to establish consistency of process performance across the organization.

Organizational implications

- a) Process management starts to occur using defined documented processes, with mandatory process objectives, and ensures that these objectives are appropriately addressed.

Level 4 - Managed

It is characteristic of processes at this level that, using process metrics, management can effectively control the AS-IS process (e.g., for software development).

Organizational implications

- a) Quantitative quality goals tend to be set for process output - e.g., software or software maintenance.
- b) Using quantitative/statistical techniques, process performance is measured and monitored and generally predictable and controllable also.

Level 5 - Optimizing

It is a characteristic of processes at this level that the focus is on continually improving process performance through both incremental and innovative technological changes/improvements.

Organizational implications

- (a) Quantitative process-improvement objectives for the organization are established, continually revised to reflect changing business objectives, and used as criteria in managing process improvement.
- (b) The effects of deployed process improvements are measured and evaluated against the quantitative process-improvement objectives.
- (c) Both the defined processes and the organization's set of standard processes are targets for measurable improvement activities.
- (d) A critical distinction between maturity level 4 and maturity level 5 is the type of process variation addressed.

Extensions

Some versions of CMMI from SEI indicate a "level 0", characterized as "Incomplete". Some pundits leave this level out as redundant or unimportant, but Pressman and others make note of it.

Comparison of CMMI Representations Staged Continuous

What is a CMMI model representation? The answer requires an explanation of the structure of CMMI models. The basic building blocks in every CMMI model are called "process areas." A process area does not describe how an effective process is executed (e.g., entrance and exit criteria, roles of participants, resources). Instead, a process area describes what those using an effective process do (practices) and why they do those things (goals).

Selecting a Representation

When making the decision about which architectural representation to use for process improvement, comparative advantages of each approach as represented in the given Table.

Continuous Representation	Staged Representation
Grants explicit freedom to select the order of improvement that best meets the organization's business objectives.	Enables organizations to have a predefined and proven path.
Enables increased visibility into the capability achieved within each individual process area.	Builds on a relatively long history of use.
Supports a focus on risks specific to individual process areas.	Case studies and data exist that show return on investment.
Affords a more direct comparison of process improvement to ISO 15504 because the organization of process areas is derived from	Permits comparisons across and among organizations.

15504.	
Allows the generic practices from higher capability levels to be more evenly and completely applied to all of the process areas.	Introduces a sequence of improvements, beginning with basic management practices and progressing through successive levels, each serving as a foundation for the next.
	Summarizes process-improvement results in a simple form a single maturity-level number.

Project Management Concerns

- Project management is the discipline of planning, organizing and managing resources to bring about the successful completion of specific project goals and objectives.
- A project is a finite endeavor (having specific start and completion dates) undertaken to create a unique product or service which brings about beneficial change or added value.
- This finite characteristic of projects stands in sharp contrast to processes, or operations, which are permanent or semi-permanent functional work to repetitively produce the same product or service. In practice, the management of these two systems is often found to be quite different, and as such requires the development of distinct technical skills and the adoption of separate management.

The traditional approach

A traditional phased approach identifies a sequence of steps to be completed. In the "traditional approach", we can distinguish 5 components of a project (4 stages plus control) in the development of a project:



Typical development phases of a project

- Project initiation stage;
- Project planning or design stage;
- Project execution or production stage;
- Project monitoring and controlling systems;
- Project completion stage.

Software Quality Assurance

Software quality assurance (SQA) consists of a means of monitoring the software engineering processes and methods used to ensure quality. The methods by which this is accomplished are many and varied, and may include ensuring conformance to one or more standards, such as ISO 9000 or CMMI.

This definition emphasizes upon three important points:

- Software requirements are the foundation from which quality is measured. Lack of conformance is lack of quality
- Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.
- A set of implicit requirements often goes unmentioned (ease of use, good maintainability etc.)

Quality Assurance

Goal of quality assurance is to provide the management with the necessary data to be informed about product quality. It consists of auditing and reporting functions of management. If data provided through QA identifies problems, the management deploys the necessary resources to fix it and hence achieves desired quality control.

Cost of quality

A very significant question is: does quality assurance add any value. That is, is worth spending a lot of money in quality assurance practices? In order to understand the impact of quality assurance practices, we have to understand the cost of quality (or lack thereof) in a system.

If we try to prevent problems, obviously we will have to incur cost. This cost includes:

- Quality planning
- Formal technical reviews
- Test equipment
- Training

SQA Activities

There are two different groups involved in SQA related activities:

- Software engineers who do the technical work
- SQA group who is responsible for QA planning, oversight, record keeping, analysis, and reporting.

SQA Group Activities

An SQA plan is developed for the project during project planning and is reviewed by all stake holders. The plan includes the identification of:

- Evaluations to be performed
- Audits and reviewed to be performed

- Standards that are applicable to the project
- Procedures for error reporting and tracking
- Documents to be produced by the SQA group
- Amount of feedback provided to the software project team

Quality Control

From one project to another we want to minimize the predicted resources needed to complete a project and calendar time. This involves a series of inspection, reviews, and tests and includes feedback loop. So quality control is a combination of measurement and feedback and combination of automated tools and manual interaction.